React Events:

就像 HTML DOM 事件一樣，React 可以根據用戶事件執行操作。

React 具有與 HTML 相同的事件：單擊、更改、鼠標懸停等。

添加事件

React 事件以駝峰式語法編寫：onClick 而不是 onclick.

React 事件處理程序寫在大括號內：

onClick={shoot} 而不是 onclick="shoot()".

React:

```
<button onClick={shoot}>Take the Shot!</button>
```

HTML:

```
<button onclick="shoot()">Take the Shot!</button>
```

範例: 按鈕點擊

```
<!DOCTYPE html>
<html>

<head>
	<meta charset="utf-8" />
	<title>Hello React Components!</title>
	<script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
	<script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
	<script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>

</head>

<body>
	<div id="root"></div>

	<script type="text/babel">
		function Football() {
			const shoot = () => {
				alert("Great Button Event!");
			}
```

```
            return (
                <button onClick={shoot}>Take the shot!</button>
            );
        }
        const root = ReactDOM.createRoot(document.getElementById('root'));
        root.render(<Football />);

    </script>
</body>
</html>
```

傳遞參數

要將參數傳遞給事件處理程序，請使用箭頭函數。

```
<!DOCTYPE html>
<html>

<head>
    <meta charset="utf-8" />
    <title>Hello React Components!</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
    <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
    <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>

</head>

<body>
    <div id="root"></div>

    <script type="text/babel">
        function Football() {
            const shoot = (a) => {
                alert(a);
            }
```

```
            return (
                <button onClick={() => shoot("Args!")}>Take the shot!</button>
            );
        }
        const root = ReactDOM.createRoot(document.getElementById('root'));
        root.render(<Football />);
    </script>
</body>
</html>
```

React List:

```
<!DOCTYPE html>
<html>

<head>
    <meta charset="utf-8" />
    <title>Hello React Components!</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
    <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
    <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>

</head>

<body>
    <div id="root"></div>

    <script type="text/babel">
        function Car(props) {
            return <li>I am a {props.brand}</li>;
        }

        function Garage() {
            const cars = ['Ford', 'BMW', 'Audi'];
            return (
```

```
                    <div>
                        <h1>In my garage:</h1>
                        <ul>
                            {cars.map((car) => <Car brand={car} />)}
                        </ul>
                    </div>
                );
            }

            const root = ReactDOM.createRoot(document.getElementById('root'));
            root.render(<Garage />);
        </script>
</body>
</html>
```

React List Keys

React key 可以跟踪元素。這樣如果一個元素被更新或刪除，只有那個元素會被重新渲染，而不是整個列表。

```
<!DOCTYPE html>
<html>

<head>
    <meta charset="utf-8" />
    <title>Hello React Components!</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
    <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
    <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>

</head>

<body>
    <div id="root"></div>

    <script type="text/babel">
```

```
        function GroceryList() {
            const items = [
                { id: 1, name: 'bread' },
                { id: 2, name: 'milk' },
                { id: 3, name: 'eggs' }
            ];

            return (
                <div>
                    <h1>Grocery List</h1>
                    <ul>
                        {items.map((item) =>
                            <li key={item.id}>{item.name}</li>)}
                    </ul>
                </div>
            );
        }
        const root = ReactDOM.createRoot(document.getElementById('root'));
        root.render(<GroceryList />);
    </script>
</body>
</html>
```

React Forms

在 React 中添加表單

您可以添加任何元素使用表單：

處理表單: 當數據更改值或被提交時如何處理數據。

1. 在 HTML 中，表單數據通常由 DOM 處理。
2. 在 React 中，表單數據通常由組件處理。

當組件處理數據時，所有數據都存儲在組件狀態中。

onChange 您可以通過在屬性中添加事件處理程序來控制更改 。

```
<!DOCTYPE html>
<html>

<head>
    <meta charset="utf-8" />
```

```
    <title>Hello React Components!</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
    <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
    <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>

</head>

<body>
    <div id="root"></div>

    <script type="text/babel">

        function MyForm() {
            const [name, setName] = React.useState("");

            const handleSubmit = (event) => {
                event.preventDefault();
                alert(`The name you entered was: ${name}`)
                }

            return (
                <form onSubmit={handleSubmit}>
                    <label>Enter your name:
                        <input
                            type="text"
                            value={name}
                            onChange={(e) => setName(e.target.value)}
                        />
                    </label>
                    <input type="submit" />
                </form>
            )
        }

        const root = ReactDOM.createRoot(document.getElementById('root'));
```

```
        root.render(<MyForm />);
    </script>
</body>
</html>
```

多個輸入字串

name 您可以通過向每個元素添加一個屬性來控制多個輸入字段的值 。

要訪問事件處理程序中的字串，請使用 event.target.name 及 event.target.value 語法。

要更新狀態，請在屬性名稱周圍使用方括號 [ ]。

```
<!DOCTYPE html>
<html>

<head>
    <meta charset="utf-8" />
    <title>Hello React Components!</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
    <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
    <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>

</head>

<body>
    <div id="root"></div>

    <script type="text/babel">
        function MyForm() {
            const [inputs, setInputs] = React.useState({});

            const handleChange = (event) => {
                const name = event.target.name;
                const value = event.target.value;
                setInputs(values => ({ ...values, [name]: value }))
            }
```

```
            const handleSubmit = (event) => {
                event.preventDefault();
                console.log(inputs);
            }

            return (
                <form onSubmit={handleSubmit}>
                    <label>Enter your name:
                        <input
                            type="text"
                            name="username"
                            value={inputs.username || ""}
                                                    // 防止出現 undefined
                            onChange={handleChange}
                        />
                    </label><p />
                    <label>Enter your age:
                        <input
                            type="number"
                            name="age"
                            value={inputs.age || ""}
                            onChange={handleChange}
                        />
                    </label>
                    <input type="submit" />
                </form>
            )
        }

        const root = ReactDOM.createRoot(document.getElementById('root'));
        root.render(<MyForm />);
    </script>
</body>
</html>
```

表單 TextArea

```
 <!DOCTYPE html>
<html>

<head>
	<meta charset="utf-8" />
	<title>Hello React Components!</title>
	<script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
	<script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
	<script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>

</head>

<body>
	<div id="root"></div>

	<script type="text/babel">

		function MyForm() {
			const [textarea, setTextarea] = React.useState(
				"textarea goes in the attribute"
			);

			const handleChange = (event) => {
				setTextarea(event.target.value)
			}

			return (
				<form>
					<textarea value={textarea} onChange={handleChange} />
				</form>
			)
		}
```

```
        const root = ReactDOM.createRoot(document.getElementById('root'));
        root.render(<MyForm />);

    </script>
</body>

</html>
```

Form Select

React 中的下拉列表或選擇框也與 HTML 有點不同。

在 HTML 中，下拉列表中的選定值是使用以下 selected 屬性定義的：

```
HTML:
<select>
  <option value="Ford">Ford</option>
  <option value="Volvo" selected>Volvo</option>
  <option value="Fiat">Fiat</option>
</select>
```

在 React 中，選擇的值是用標籤 value 上的一個屬性定義的

```
<!DOCTYPE html>
<html>

<head>
    <meta charset="utf-8" />
    <title>Hello React Components!</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
    <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
    <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>

</head>

<body>
```

```
    <div id="root"></div>

    <script type="text/babel">
        function MyForm() {
            const [myCar, setMyCar] = React.useState("Volvo");

            const handleChange = (event) => {
                setMyCar(event.target.value)
            }

            return (
                <form>
                    <select value={myCar} onChange={handleChange}>
                        <option value="Ford">Ford</option>
                        <option value="Volvo">Volvo</option>
                        <option value="Fiat">Fiat</option>
                    </select>
                </form>
            )
        }
        const root = ReactDOM.createRoot(document.getElementById('root'));
        root.render(<MyForm />);
    </script>
</body>
</html>
```

React Router

範例 1

```
<!DOCTYPE html>
<html>

<head>
    <meta charset='UTF-8'>
    <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
    <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
<script
src="https://unpkg.com/react-router-dom@5.3.2/umd/react-router-dom.min.js">
</script>

    <script src='https://unpkg.com/babel-standalone@6.26.0/babel.js'></script>
</head>

<body>
    <div id='root'></div>
    <script type='text/babel'>
        const Link = ReactRouterDOM.Link;
        const Route = ReactRouterDOM.Route;

        const App = () => (
            <ReactRouterDOM.HashRouter>
                <ul>
                    <li><Link to="/">Home</Link></li>
                    <li><Link to="/login">Login</Link></li>
                    <li><Link to="/register">Register</Link></li>
                </ul>

                <Route path="/" exact component={Home} />
                <Route path="/login" component={Login} />
                <Route path="/register" component={Register} />
            </ReactRouterDOM.HashRouter>
```

```
            )

            const Home = () => <h1>Home</h1>
            const Login = () => <h1>Login</h1>
            const Register = () => <h1>Register</h1>

            ReactDOM.render(<App />, document.querySelector('#root'));
        </script>
</body>

</html>
```

範例 2

```
<!DOCTYPE html>
<html>

<head>
        <meta charset='UTF-8'>
        <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
        <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
        <script src='https://unpkg.com/babel-standalone@6.26.0/babel.js'></script>

        <script
src="https://unpkg.com/react-router-dom@5.3.2/umd/react-router-dom.min.js"></s
cript>
</head>

<body>
        <div id='root'></div>
        <script type='text/babel'>
            const Home = () => {
                return <h1>Home Func</h1>;
            };

            const Blogs = () => {
                return <h1>Blog Articles</h1>;
```

```
};

const Contact = () => {
    return <h1>Contact Me</h1>;
};
const NoPage = () => {
    return <h1>404 Not Page</h1>;
};
const Layout = () => {
    return (
        <div>
            <nav>
                <ul>
                    <li>
                        <Link to="/">Layout</Link>
                    </li>
                    <li>
                        <Link to="/home">Home</Link>
                    </li>
                    <li>
                        <Link to="/blogs">Blogs</Link>
                    </li>
                    <li>
                        <Link to="/contact">Contact</Link>
                    </li>
                </ul>
            </nav>

        </div>
    )
};
const Link = ReactRouterDOM.Link;
const Route = ReactRouterDOM.Route;
function App() {
    return (
        <ReactRouterDOM.BrowserRouter>
            <Route path="/" extract component={Layout} />
            <Route path="/home" component={Home} />
```

```
                        <Route path="/blogs" component={Blogs} />
                        <Route path="/contact" component={Contact} />
                    </ReactRouterDOM.BrowserRouter>
                );
            }
            const root = ReactDOM.createRoot(document.getElementById('root'));
            root.render(<App />);
        </script>
</body>

</html>
```

React Using CSS

Inline-CSS: (內聯樣式)

注意：在 JSX 中 JavaScript 表達式寫在大括號內，由於 JavaScript 物件也使用大括號，所以上例中的樣式寫在兩組大括號內{{}}

```
<!DOCTYPE html>
<html>

<head>
    <meta charset="utf-8" />
    <title>Hello React Render!</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
    <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
    <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>
</head>
<body>
    <div id="root"></div>
    <script type="text/babel">
        const Header = () => {
            return (
                <div>
                    <h1 style={{ color: "red" }}>Hello Style!</h1>
                    <p>Add a little style!</p>
                </div>
            );
        }

        const root = ReactDOM.createRoot(document.getElementById('root'));
        root.render(<Header />);

    </script>
</body>
</html>
```

建一個名稱為 myStyle 的樣式物件：

```
<!DOCTYPE html>
<html>
<head>
	<meta charset="utf-8" />
	<title>Hello React Render!</title>
	<script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
	<script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
	<script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>
</head>

<body>
	<div id="root"></div>
	<script type="text/babel">
		const Header = () => {
			const myStyle = {
				color: "blue",
				backgroundColor: "DodgerBlue",
				padding: "10px",
				fontFamily: "Sans-Serif"
			};
			return (
				<div>
					<h1 style={myStyle}>Hello Style!</h1>
					<p>Add a little style!</p>
				</div>
			);
		}
		const root = ReactDOM.createRoot(document.getElementById('root'));
		root.render(<Header />);
	</script>
</body>
</html>
```

您可以在單獨的文件中編寫 CSS 樣式，只需使用 .css 文件附檔名保存該文件，然後將其導入到您的應用程序中。

App.css

```
body {
    background-color: #282c34;
    color: white;
    padding: 40px;
    font-family: Sans-Serif;
    text-align: center;
}
```

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Hello React Render!</title>
    <link rel="stylesheet" href="App.css">
    <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
    <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
    <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>
</head>
<body>
    <div id="root"></div>
    <script type="text/babel">
        const Header = () => {
            return (
                <div>
                    <h1>Hello Style!</h1>
                    <p>Add a App.css file style!.</p>
                </div>
            );
        }
        const root = ReactDOM.createRoot(document.getElementById('root'));
        root.render(<Header />);
    </script>
```

```
</body>
</html>
```